让 CANN 知识成为 Agent 的生产力

cannbot-knowledge

开源知识库实践与共建

让经过验证的工程经验，持续服务下一次任务



CANN 开源社区 · 技术分享

https://gitcode.com/cann

CANN

目录

从工程问题到共建实践

01 **背景与挑战**

从分散资料到共享工程知识

02 **知识库架构**

知识如何组织、浏览与追溯

03 **知识库使用与效果**

安装、使用场景与实测效果

04 **知识生产与治理**

知识提取、检索与审核

05 **后续规划**

知识覆盖、质量评估与开放共建

痛点一：知识分散在各个 Skill，复用与更新成本上升

01 背景与挑战 / 从分散维护到共享复用

历史仓库统计　150 / 239 个 Skill 携带 references/ 参考资料 **62.8%**



Skill 负责流程与验收，知识库承载可复用、可扩展的工程事实

痛点二：读懂源码，还需还原设计与适用边界

以官方源码为参考的 4 个 L2 算子实验中，仅 Gather 完成交付闭环

Claude Code / GLM-5.2 · 各算子独立工作区 · 每算子最多 10 次上板测试

Gather	Softmax	Scatter	Cummin
正确性、性能与交付闭环	精度未通过	CPU 执行违规，得分为 0	精度未通过

知识卡串联实现链路，解释路线选择与关键约束

接口入口 → Host 调度 → Tiling 切分 → Device Kernel

平台与输入条件
内存与同步
构建与版本边界

固定提交源码提供证据，知识卡片包含设计、约束与适用条件

CANN

痛点三：经验留在一次会话里，后续任务仍会重复试错

原始开发轨迹

- 多轮试错与无效猜测
- 报错、调用与代码变更
- 环境信息与潜在敏感内容

会话结束，经验容易流失

→

受约束提炼

相关性筛选

机制与证据还原

跨会话归并

独立证据核验

→

结构化经验卡 • Runbook

现象 遇到什么异常	**信号** 哪些日志可识别
根因 为什么会发生	**解法** 哪些修改已验证
边界 适用平台与版本	**验证** 通过哪些测试

保留关键代码与验证结果，让调试经验能被后续任务复用

CANN

业界参考：持续维护知识、统一格式、按范围检索

LLM Wiki

让知识持续复利

原始来源 → 结构化 Wiki

Ingest / Query / Lint 闭环

维护主题与关联，处理重复与矛盾

落点 持续维护知识资产
补充确定性机器契约

OKF · 开放知识格式

让知识可读、可交换

Markdown + Frontmatter

目录 Bundle 与普通链接

来源、信任与生命周期

落点 采用 OKF v0.2
叠加 CANNBot Profile 约束

KernelWiki

让领域知识进入工作流

来源 / 知识 / 查询分层

固定 revision，限定范围

检索后继续阅读正文与来源

落点 按技术与平台检索
让判断回到可核验证据

统一规则贯穿知识生产、质量检查与查询消费

渐进式披露：先定位知识，再逐层读到证据

01 目录与索引
在哪个范围找？
按领域、Profile、目标平台缩小候选集

02 摘要与 Frontmatter
这张卡适用吗？
看标题、摘要、平台和状态，判断相关性

03 知识正文
结论与条件是什么？
阅读全文，核对接口约束、实现机制和失效边界

04 原始证据
来源依据能否复核？
打开登记来源；需要更多上下文时再读关联卡

分层目录、索引、摘要帮助选卡，正文支持判断，原始来源支持核验

CANN

打开仓库：一张知识卡的阅读路径

02 知识库架构 / 目录结构与真实内容摘录

CANN社区开源知识库 cannbot-knowledge

gitcode.com/cann/cannbot-knowledge

cannbot-knowledge/

knowledge/
  index.md
  common/
  contrib/
  ops/ascendc/
    apis/
    operators/
    runbooks/
  graph/ model/ runtime/

governance/ 治理契约

.agents/skills/ 生产与查询

docs/ recipes/ 使用指南

唯一知识包：knowledge/

1 index.md 选择知识卡

[CeilDivision](ceil_division.md)
计算两个整数相除后向上取整结果。

2 Frontmatter 判断适用性

type: API
title: CeilDivision
status: stable
platforms: [950, a2, a3]

3 正文读取签名与约束

CeilDivision(num1, num2)
返回向上取整的整数结果。

约束：仅支持 num1、num2 为正数
num2 为 0 时，结果为 0

4 sources.resource 定位原文

cann-docs-raw/
  asc-devkit/docs/zh/api/
  Utils-API/cpp_stdlib/
  math_functions/CeilDivision.md

按来源身份恢复资源，再核对原始语料

9058 张知识卡，覆盖多类开发场景

02 知识库架构 / 知识规模与领域覆盖

一级领域分布



算子开发
7,416 张　81.87%

社区贡献
581 张　6.41%

模型
525 张　5.80%

Runtime
515 张　5.69%

公共知识
21 张　0.23%

二级目录分布

算子开发　7,416 张



Ascend C	6,780	91.42%
Triton	497	6.70%
CATLASS DSL	97	1.31%
CATLASS	37	0.50%
TileLang	4	0.05%

模型　525 张



推理 inference	403	76.76%
训练 training	86	16.38%
具身 embodied	22	4.19%
空间 spatial	14	2.67%

三条命令，把知识库接入自己的项目


```
git clone --branch master https://gitcode.com/cann/cannbot-knowledge.git
cd cannbot-knowledge
bash install.sh claude /path/to/project consumer
```

/path/to/project 换成已有项目目录；claude 可换为 codex、opencode、trae、cursor 或 copilot。

安装完成后，按三步开始使用

1 确认安装位置

确认安装输出指向
你的项目与知识仓

2 打开项目内 Agent

在自己的业务项目中
打开或重启 Agent 会话

3 完成一次查询

先看覆盖，再查具体问题
检查回答是否附有已读来源

接入后，在 Agent 中描述任务，让知识检索 Skill 帮你找到依据。

准备：Bash、Git、Python ≥ 3.10、已有项目目录；首次安装需联网。

CANN

怎么用：说清任务，让 Agent 带着知识完成工作

03 知识库使用与效果 / 从提问到验证

1 交代任务上下文

平台、版本与任务目标
已有代码或首发报错

2 指定先查知识

调用查询 Skill 阅读正文
核对当前任务是否适用

3 要求依据与验证

说明采用了什么知识
附来源与实际验证结果

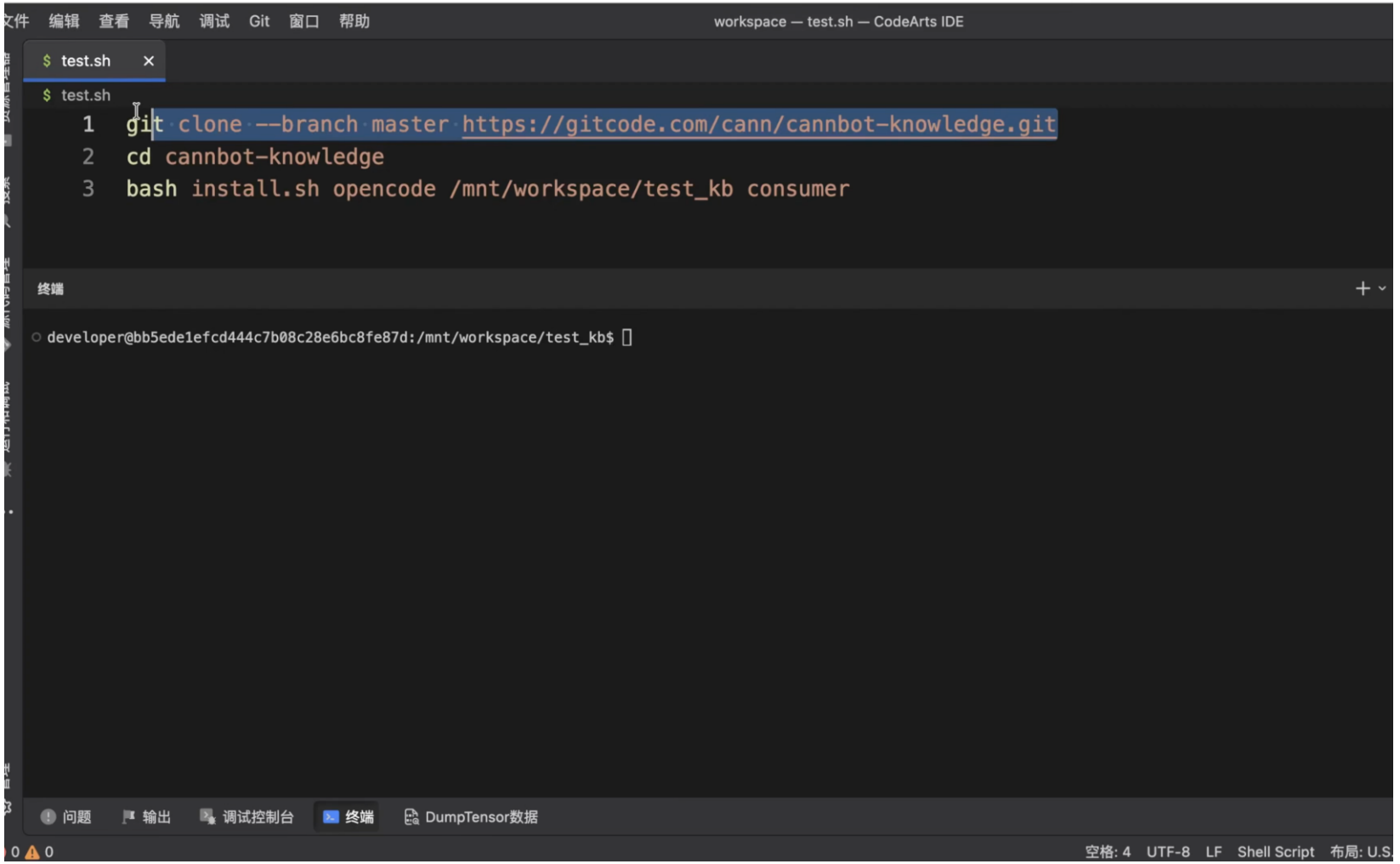
可以直接这样向 Agent 提问

我在 950 上实现 Softmax。
请先用 knowledge-query 查相关设计与 API 约束，再给出实现方案。
出现新报错时，查相似排障经验；交付时附来源与测试结果。

编码前查设计 • 遇到新问题再查 • 最后在当前工程验证

三条命令，把知识库接入自己的项目

03 知识库使用与效果 / 安装演示



https://gitcode.com/cann

CANN

三个场景：查接口、排故障、做模型优化

现有知识的用法示例

查接口，少翻资料

DataCopyPad 的参数怎么填?

怎样用

950 上非对齐搬运怎么写?
请核对参数单位与限制，
并给出已读来源。

知识库提供

API 签名、参数与平台限制
附可核对的官方来源

明确两侧步长的单位
减少参数误用

排故障，少走弯路

编译成功，上板却运行失败

怎样用

950 上运行报 507035，
请查相似经验，
给出排查顺序。

知识库提供

相似排障经验与历史处置
提示核对构建目标

把构建配置纳入排查
避免只盯计算逻辑

做优化，先判断适用性

服务重启后，首次推理慢

怎样用

能否复用图编译结果?
请结合当前后端与版本，
判断缓存方案是否适用。

知识库提供

编译缓存方案及复用方式
后端限制与缓存失效条件

找到已有方案和使用边界
再安排目标环境验证

以上是用法示例；下一页展示 API 查询与算子开发的实测对照。

CANN

其他 Skill 如何用：把共享知识嵌入任务流程

03 知识库使用与效果 / 三种工作流接入方式

参考工作流接入示例

业务 Skill • 什么时候查	知识怎样进入流程	下游如何使用
算子开发 **ops-registry-invoke** 设计与编码开始前	→ **查询算子语义、API 与平台约束** 阅读卡片，整理为 preflight.md	→ **后续设计 / 开发节点读取备忘** 带着约束、已知问题与出处实现
迁移与适配 **ascendc-port-orchestrator** 分派子 Agent 任务前	→ **编排器检索外部知识卡** 卡片引用注入 brief，正文只读访问	→ **执行 Agent 带着知识完成子任务** 据此实现、排错，再编译与测试
Triton 性能优化 **triton-optimizer** 已有实测症状，需要选方案时	→ **按症状路由到优化模式 / 优化点卡** 索引定位，正文来自共享知识库	→ **筛选候选方法，实施后重新测量** 知识提供方向，效果由实测确认

同一份 cannbot-knowledge：可查询选卡，也可把卡片引用送入下游任务。

CANNBot 实践1：API 查询开销下降，共同算子总分提高

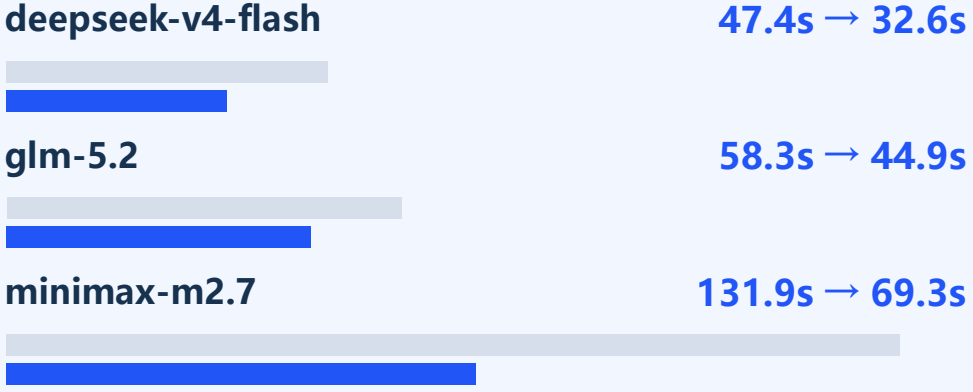
03 知识库使用与效果 / 报告中的对照结果

75 条真实 API 查询 • 3 种模型分别对照

耗时下降约 23%～47%

检索内容重合率 >96%

仅比较同模型同轮：逐文件查文档 → 知识库查询



灰：原始文档检索　蓝：知识库查询；横条采用同一秒数刻度

特定实验口径；GLM-5.3 有 12 个算子提升、4 个下降；总分提升不等于运行加速。

CANNBench：共同算子的合计得分

GLM-5.2 • 11 算子 **+13.40%**

1275.42 → 1446.30 分

GLM-5.3 • 16 算子 **+7.39%**

2057.39 → 2209.43 分

Micro-Official-Tasks 1.1.0 · 950pr

总分口径：公开分 + 隐藏分

CANNBot 实践2：算子生成跳过探针实验，UT 交互轮数下降近半

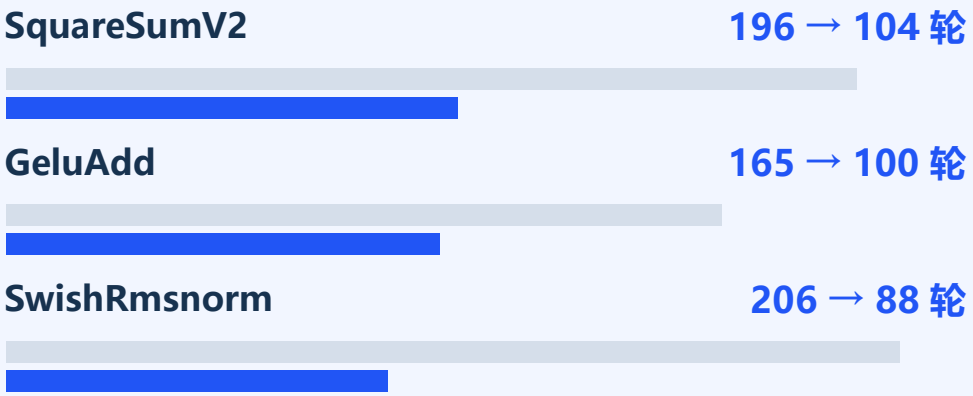
03 知识库使用与效果 / 知识库接入CANNBot 官方插件 ops-registry-invoke

Host Tiling UT 开发 • 3 个算子对照

轮数下降约 39%～57%

两组均通过验收

GLM-5.3-Flash 同起点：原始工作流 → 知识库辅助



灰：原始工作流　蓝：知识库辅助；横条采用同一轮数刻度

ops-registry-invoke：探针实验可跳过

接入前 • 探针实验

多轮探针

writing tiling design 需实验摸清 API 用法

接入后 • 知识卡直达

省约 20 分钟

API 语义与约束检索即得，实验跳过

工作流：ops-registry-invoke
收益口径：writing tiling design 节点 · 单算子实测

左图 UT 实验为典型成功对照、非重复实验平均，轮数含执行与验证；右图 20 分钟为 writing tiling design 节点单算子实测。

三种知识提取 Skill：从材料到可复用知识

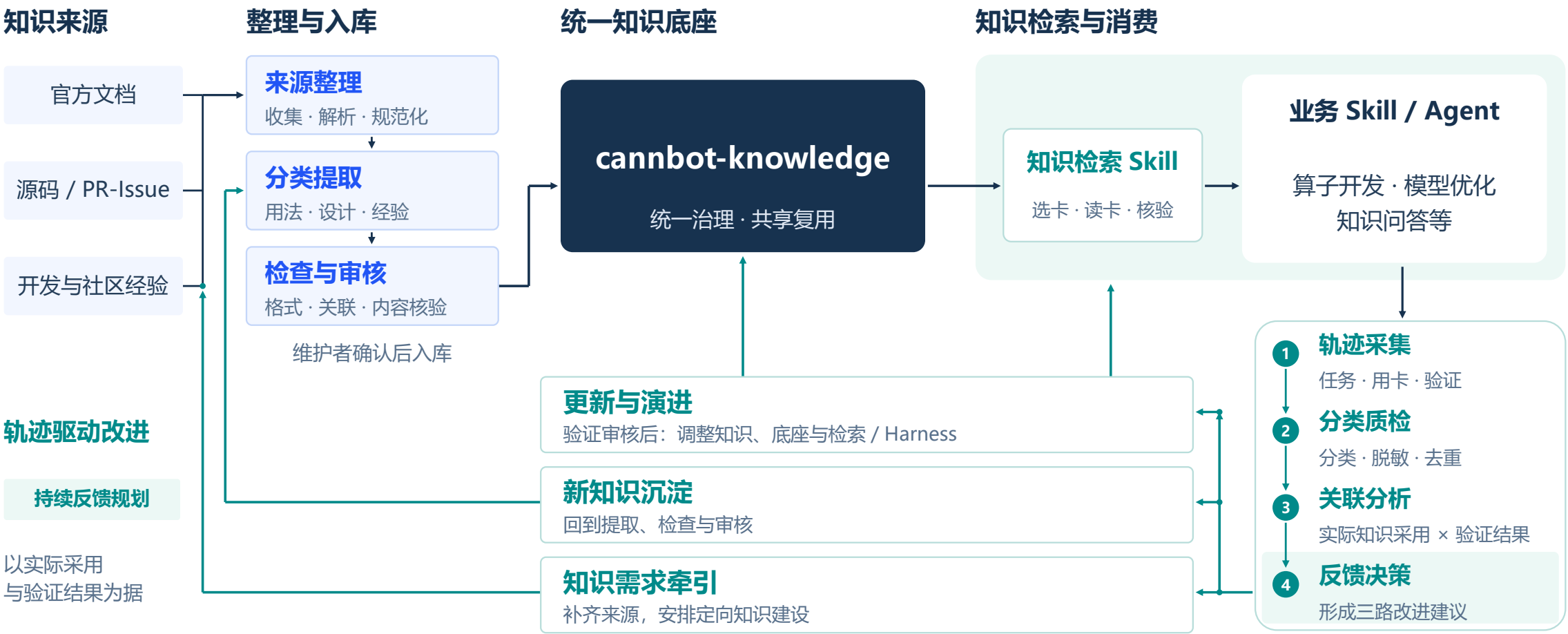
输入材料	怎么提取			产出
官方文档 knowledge-ingest	**定位来源并查重** 确认文档版本和原文，查找已有知识卡。	**整理技术内容** 保留接口、约束和示例；跨页内容按证据归纳。	**补齐字段并检查** 标明平台和出处，核对链接与检索结果。	**API / 指南卡** 查接口用法和限制
算子源码 VV / CV 源码提取	**锁定源码与入口** 固定代码版本，确认目标平台和算子入口。	**追踪实际执行** 沿 Host 调度和切分，找到设备端实现。	**提炼可复用方案** 按执行路线整理计算、数据搬运与同步。	**算子实现卡** 参考同类算子的写法
Agent 实践轨迹 trace-ingest	**解析与脱敏** 将会话整理成事件流，保留原始记录的位置。	**提取问题与动作** 找出报错、诊断步骤，还原关键修改。	**核对证据并入库** 用工具结果核验结论，去重并标明适用条件。	**排障经验卡** 参考诊断和修复步骤

三类知识入库前都要说清：依据来自哪里，什么条件下可以复用。

CANN

知识闭环：多源生产、按需使用、轨迹驱动改进

04 知识生产与治理 / 检索、入库与持续改进的整体架构



https://gitcode.com/cann

CANN

下一步：知识更全、查得更准、用得更好

05 后续规划 / 社区持续贡献，真实任务检验

后续规划

01	**扩大知识覆盖**	补齐常用接口、算子开发、模型优化与故障经验。
02	**看清知识质量**	用健康度看板发现缺口、过时内容与效果变化。
03	**用反馈改进知识**	从真实任务提炼新经验，审核后持续修订。
04	**让检索更精准**	优化检索与多路召回，更快找到适用知识。
05	**探索 MCP 服务**	让更多 Agent 访问同一份知识与来源。

欢迎从一个真实问题、一条可核查经验、一次使用反馈开始共建。

https://gitcode.com/cann

CANN

让一次成功，
成为可复用的能力

让 CANN 知识成为 Agent 的生产力

从一次查询、一个知识缺口、一条可复核经验开始共建

gitcode.com/cann/cannbot-knowledge

感谢聆听 · 欢迎社区交流

社区愿景：打造开放易用、技术领先的AI算力新生态
社区使命：以开源驱动技术创新，降低开发门槛，聚力构筑高性能昇腾 AI 算力底座

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem
Mission: Drive technological innovation through open source, lower development barriers, and jointly build a high-performance Ascend AI computing infrastructure.

上CANN社区获取干货

关注CANN公众号获取资讯

CANN